Implementation Document

for

IncognIlTo

Version 1.0

Prepared by

Group 7

Group Name:
Code Commanders

Arsh Aryan	230203	arsharyan23@iitk.ac.in
Abeer Gupta	230031	abeergupta23@iitk.ac.in
Joel Bansal	230509	joelb23@iitk.ac.in
Krish Arora	230571	krishar23@iitk.ac.in
Mahir Jain	230618	mahirj23@iitk.ac.in
Harsh Wardhan Singh	230453	harshwss23@iitk.ac.in-
Archit Kumar Gupta	230186	architkg23@iitk.ac.in
Krish Vijay Tiwari	230573	krishvt23@iitk.ac.in
Om Bhartiya	230714	ombhartiya23@iitk.ac.in
Prakhar Kaushal	230768	prakhark23@iitk.ac.in

Course: CS253

Mentor TA: Imran Shaik

Date: 29/03/26

Contents

CONTENTS II
REVISIONS III

1 IMPLEMENTATION DETAILS 4

2 CODEBASE 9

3 COMPLETENESS 25

APPENDIX A - GROUP LOG 28

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Om Bhartiya Joel Bansal Krish Arora Krish Vijay Tiwari Prakhar Kaushal Abeer Gupta Arsh Aryan Archit Kumar Gupta Mahir Jain Harsh Wardhan Singh Shekhawat	First version of the implementation document	19/03/26

1 Implementation Details

IncognIITo implements a three-unit architecture, a popular pattern for user-interactive applications.

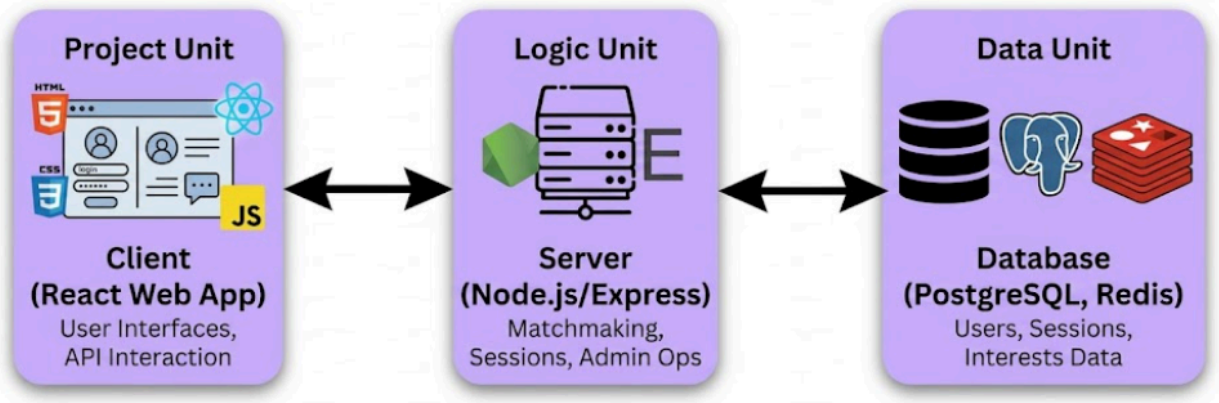
This design ensures clear separation of concerns between user interaction, application logic, and data management.

The Units this architecture comprises of include:

● **Project Unit:** This represents the components users directly interact with. In our case, this is a web application built using HTML, CSS, and JavaScript (with React) that runs in the browser. It provides interfaces for login, matchmaking, chat/video interaction, and the admin dashboard, and communicates with the backend through API calls.

● **Logic Unit:** This contains the code required to translate user actions into application functionality, implemented using Node.js and Express. This layer handles API requests, manages matchmaking, controls the creation and termination of sessions, banning users, unbanning users, managing interest tags, and reviewing and resolving user reports. It serves as the core layer that processes requests and connects the frontend with the database.

● **Data Unit:** This consists of the PostgreSQL database that stores all relevant data for the application, including user information, session details, and interests. It supports backend operations such as session tracking, user management, and enforcing restrictions like banned users, ensuring consistency and reliability of the system.



1.1 Project Unit

The project unit forms the interface through which users interact with the system via a web browser. It is responsible for capturing user inputs, displaying system responses, and ensuring smooth navigation across different functionalities. A well-designed interface is essential for enabling users to access features like matchmaking, chatting, and administrative controls in an intuitive manner.

The following choices allow efficient handling of different views such as authentication pages, matchmaking screens, live interaction interfaces, and the admin dashboard. The component-based approach improves maintainability and ensures smooth updates during real-time interactions.

Programming Languages:

- **HTML & CSS:** Used for structuring and styling the web interface.
- **JavaScript:** Used for implementing interactive behavior in the browser.
- **TypeScript:** Used across both frontend and backend to enforce type safety and reduce runtime errors. It ensures structured handling of API requests, database responses, and real-time socket events, improving code reliability and maintainability. This is especially important in a system like IncognIITo where multiple components (authentication, chats, matchmaking) interact dynamically

Framework:

- **React:** Used for building dynamic and reusable user interface components.

Libraries and Tools:

- **Socket.IO (client):** Used for real-time communication such as matchmaking events, session updates, and messaging.
- **WebRTC:** Used for enabling peer-to-peer video communication between users.
- **Fetch API:** Used for making asynchronous HTTP requests to the backend.
- **Cloudinary:** Used for storing and serving user profile images.
- **Tailwind CSS:** Used for utility-first styling across all frontend components, enabling rapid UI development with consistent design tokens.
- **Vite:** Used as the frontend build tool for fast development server startup and optimized production bundling.

Justification of Choices:

- **React** was chosen for its component-based architecture, which improves reusability and maintainability. Its Virtual DOM ensures efficient rendering, making it suitable for interactive UI flows.
- **Socket.IO** enables real-time bidirectional communication, which is essential for handling matchmaking, session lifecycle updates, and messaging.

- **WebRTC** allows direct peer-to-peer video communication without routing media through the server, ensuring low latency and preserving user privacy.
- **Vite** was chosen as the frontend build tool for its significantly faster development server startup and hot module replacement compared to alternatives like Webpack.

- **Cloudinary** was used for storing and serving user profile images externally rather than on the application server, eliminating the need to handle file storage and CDN delivery in-house.
- **Tailwind CSS** eliminates the need for custom CSS files by providing utility classes directly in markup, significantly speeding up UI development while maintaining consistency across components.

1.2 Logic/Reasoning Unit

The logic unit acts as the core processing layer of the system, responsible for handling requests received from the frontend, executing application logic, and coordinating data flow between different components. It ensures that user actions are correctly interpreted and translated into appropriate backend operations.
This layer is responsible for implementing key functionalities such as user authentication, matchmaking between users, and managing session lifecycles. It handles the creation of sessions when users are matched, maintains active connections during interactions, and ensures proper cleanup when sessions end.
In addition, the backend supports administrative control operations, including retrieving user information, banning users, unbanning users, and reviewing and resolving user reports. These operations are executed through role-protected API endpoints accessible only to administrators.

Programming Languages:

- **JavaScript (Node.js):** Used for server-side development.

Framework:

- **Express.js:** Used for building RESTful APIs and handling request routing.

Libraries and Tools:

- **Socket.IO:** Used for real-time communication such as matchmaking, session lifecycle events, and messaging.
- **JWT (JSON Web Tokens):** Used for stateless authentication and session management.
- **bcrypt:** Used for secure password hashing and storage.
- **OTP/Email Service:** Used for IITK email verification during registration and password recovery.
- **Middleware:** Used for authentication validation and role-based access control (admin vs user).

- **Redis:** Used for managing the matchmaking queue, caching user interest bitmaps, and storing active session state for real-time matching operations.

Justification of Choices:

- **Node.js** was chosen for its non-blocking, event-driven architecture, making it suitable for handling multiple concurrent users and real-time interactions efficiently.
- **Express.js** provides a lightweight and flexible framework for building APIs, allowing clear organization of routes and middleware.
- **Socket.IO** enables real-time bidirectional communication, which is essential for matchmaking, session management, and in-session interactions.
- **JWT + bcrypt** ensures secure authentication by enabling stateless session management and safe password storage.
- **Redis** was chosen for managing the matchmaking queue and active session state due to its in-memory data model, which provides microsecond-level read/write performance.

1.3 Data Management Unit

The data management unit is responsible for storing, organizing, and maintaining all information required by the application. It acts as the persistent storage layer, ensuring that user data, session details, and system state are reliably recorded and can be efficiently retrieved when needed.

Database System:

- **PostgreSQL:** A relational database system used to store and manage structured application data such as users, sessions, interests, reports, and messages. It serves as the data layer responsible for maintaining consistency and persistence of information in the system.
- **Redis:** Used as an in-memory data store for three distinct purposes: managing the matchmaking queue as a sorted set ordered by join time, caching user interest bitmaps to avoid repeated database lookups during the matching cycle, and storing active session mappings to track which room each matched user has been assigned to. Its sub-millisecond read/write performance makes it essential for the real-time matching loop that runs every second.
- **Cloudinary:** Used for storing and serving user profile images externally.

Justification of Choices:

- **PostgreSQL** provides strong reliability through ACID-compliant transactions, ensuring data integrity for critical operations such as authentication, session management, and reporting.
- It supports high scalability and efficient handling of concurrent users, making it suitable for a system with real-time interactions and large numbers of simultaneous sessions as specified in the SRS.
- Its relational data model is well-suited for managing structured and interconnected data such as users, sessions, interests, and reports in a consistent and organized manner.

- **Redis** was chosen over a traditional database for real-time state management because its in-memory architecture eliminates the latency overhead of disk I/O, which is critical for a matching loop that evaluates the entire queue every second. Its sorted data structure maps naturally to a time-ordered queue, and its key-value model is ideal for fast session lookups and interest bitmap caching.
- **Cloudinary** was used to store media assets such as profile images externally, reducing load on the primary database.

1.4 Development and Version Control Environment

We used **Git** as our version control system to track changes and maintain the history of the codebase. GitHub was used for repository hosting and collaboration among team members. Development was coordinated using branches, commits, and merges across different components of the project. A task distribution plan was followed, and a WhatsApp group was used for communication and coordination.

1.5 Authentication and Authorization

The system's approach to authentication and authorization is multi-layered, focusing on controlled access, stateless session management, and user anonymity:

- **Secure Registration and Access Control:** Access is strictly restricted to IITK users through institute email ID submission. Registration requires a multi-step verification process involving an OTP sent to the institute email. Upon successful validation, user passwords are secured using bcrypt hashing, while non-IITK addresses are rejected at the submission stage to prevent unauthorized entry.
- **Authentication and Stateless Session Management:** The system verifies email and password against stored credentials during login. Following successful authentication, a JSON Web Token (JWT) is issued to the user, facilitating efficient and stateless session management for all subsequent requests.
- **JWT-Based Middleware and Role-Based Authorization:** Authorization is strictly enforced via middleware that validates tokens for all protected routes, rejecting any missing, invalid, or expired tokens. Role-based access control (RBAC) further restricts sensitive administrative operations—including user banning, interest tag management, and report resolution—to authorized Administrators only.
- **User Anonymity and Data Privacy:** To preserve privacy, authentication data like email and credentials are stored separately from public profile information. All user-facing interactions utilize anonymized identifiers, ensuring that real identities are never exposed through the application.

2 Codebase

We have made a repository on GitHub through which we can store and collaborate on the source code of this project IncognIITo.

FRONTEND:

```
IncognIITo/                                # React-based UI Application
├── src/                                   # Source code directory
│   ├── app/                               # Core application logic and UI
│   │   ├── components/                    # Reusable and page-level React components
│   │   │   ├── ActiveUsersScreen.jsx      # Grid of verified users to send connection requests
│   │   │   ├── AdminDashboard.tsx         # Admin panel for user bans and report resolution
│   │   │   ├── AdminRoute.tsx             # Route wrapper restricting access to admins
│   │   │   ├── ChatRequestsDashboard.tsx  # Manage incoming/sent connection requests
│   │   │   ├── DedicatedLoginScreen.tsx   # Secure login page with role-based redirection
│   │   │   ├── ForgptPassword.tsx         # Password recovery flow with OTP
│   │   │   ├── FuturisticChatInterface.tsx # Main real-time text chat UI with sockets
│   │   │   ├── Homepage.tsx               # Primary dashboard with platform highlights
│   │   │   ├── LandingAuthPortal.tsx      # Public landing page with stats and auth links
│   │   │   ├── LiveInteractionRoom.tsx    # WebRTC room for 1:1 video/audio and side chat
│   │   │   ├── MainDashboard.tsx          # Lists all ongoing text conversations
│   │   │   ├── MatchingBuffer.tsx         # Waiting screen polling for video match queue
│   │   │   ├── OTPVerificationScreen.tsx  # Styled UI for entering 6-digit OTP codes
│   │   │   ├── PostSessionModal.tsx       # Post-call popup for ratings and reports
│   │   │   ├── ProtectedRoute.tsx         # Ensures only authenticated users can access
│   │   │   ├── PublicRoute.tsx            # Redirects logged-in users away from auth pages
│   │   │   ├── PublicUserProfile.tsx      # Read-only view of another user's profile
│   │   │   ├── RegistrationScreen.tsx     # Sign-up flow with email and OTP verification
│   │   │   ├── ThemeToggle.tsx            # Reusable button to switch light/dark modes
│   │   │   └── UserProfile.tsx            # Editable user dashboard for names, avatars, tags
│   │   ├── constants/                     # Static application data
│   │   │   └── interests.ts               # Predefined array of tags for matchmaking
│   │   ├── contexts/                      # React context providers
│   │   │   └── ThemeContext.tsx           # Global state management for light/dark themes
│   │   ├── hooks/                         # Custom React hooks
│   │   │   ├── useGlobalCleanup.ts        # Clears matchmaking queues on tab close
│   │   │   └── useThemeColors.ts          # Exposes Tailwind colors based on active theme
│   │   ├── layout/                        # Global UI layouts
│   │   │   └── AppShell.tsx               # Wrapper providing ambient backgrounds globally
│   │   ├── routes/                        # Application routing
│   │   │   └── AppRoutes.tsx              # Defines public, protected, and admin routes
│   │   ├── utils/                         # Helper utilities
│   │   │   └── theme-config.ts            # Tailwind utility class mappings for themes
│   │   └── App.tsx                        # Root component wrapping the app in providers
│   ├── services/                          # API clients and external connections
│   │   ├── auth.ts                        # Manages JWT tokens and authenticated fetch wrappers
│   │   ├── config.ts                      # Base URLs for API and Socket connections
│   │   ├── socket.ts                      # Initializes Socket.io and enforces single-session
│   │   └── user.ts                        # API helpers for profiles, avatars, and ratings
│   ├── styles/                            # Global stylesheets
│   │   ├── index.css                      # Global CSS variables and Tailwind directives
│   │   └── ...                            # Other font/theme CSS files
│   └── main.tsx                           # React DOM root render file
├── index.html                             # Main HTML template and app entry point
├── package.json                           # Project metadata, scripts, and dependencies
├── tsconfig.json                          # Primary TypeScript configuration
├── tsconfig.node.json                     # TypeScript config for Node-environment files
└── vite.config.ts                         # Vite bundler config with API/Socket proxy rules
```

After opening the **IncognIlTo** frontend folder, you can see the subsequent directory structure. It has mainly the following major parts:

src: Contains all source code for the application, organized into:

- **app/components**: Reusable React components organized by feature (e.g., Authentication, Admin Dashboard, and Futuristic Chat Interface).
- **contexts**: State management using the React Context API for global themes.
- **hooks**: Custom React hooks for logic like global cleanup and theme color exposure.
- **services**: Manages API clients, JWT tokens, and Socket.io initializations for real-time communication.
- **routes**: Defines application routing, including public, protected, and admin-only paths.
- **main.tsx**: The React DOM root render file that serves as the entry point.

index.html: The root HTML document that serves as the entry point for the application.
Configuration files: Includes `vite.config.ts` for build rules and `tsconfig.json` for TypeScript configuration

BACKEND:

The backend folder structure reveals the following major components:

- **server.ts**: The main server entry point that initializes the application and connects routes.
- **config**: Contains configuration for services like Cloudinary, Redis, and the main database connection.
- **controllers**: Business logic separated from routes for cleaner code organization, including authentication and match handling.
- **routes**: API endpoints organized by functionality such as authentication, chat, and user matching.
- **middleware**: Authentication and request processing utilities, including error handling and file upload management.
- **services**: Dedicated logic for specialized tasks like email sending, matching algorithms, and OTP management.
- **socket**: Handles real-time communication for messaging features.
- The backend follows a modular architectural pattern with RESTful API design principles. Authentication is implemented using secure protocols, while real-time communication for the messaging feature is handled through Socket integration.

Environment variables in the `.env` file configure sensitive information like database connections and API keys. The server provides comprehensive error handling with consistent response formats and meaningful error messages. File upload functionality is managed through dedicated middleware for handling user media and profile assets.The application clearly separates concerns with routes handling API endpoints, middlewares for cross-cutting concerns, and controllers and services implementing the core business logic.

```
backend/                                # Node.js & TypeScript Server
├── src/                                # Main source directory
│   ├── config/                         # External service configurations
│   │   ├── cloudinary.ts               # Configuration for image storage and CDN services
│   │   ├── database.ts                 # PostgreSQL connection pool and Sequelize/TypeORM setup
│   │   └── redis.ts                    # Redis client configuration for caching and queues
│   ├── constants/                      # Shared static data
│   │   └── interests.ts                # Static interest data and enums for user matching
│   ├── controllers/                    # Request handling logic
│   │   ├── adminController.ts          # Administrative logic for bans and report resolution
│   │   ├── authController.ts           # Authentication flow logic (Login, Signup, OTP)
│   │   └── matchController.ts          # User matching and connection request logic
│   ├── middleware/                     # Express request interceptors
│   │   ├── authMiddleware.ts           # JWT verification and role-based authorization
│   │   ├── errorHandler.ts             # Centralized global error and exception handling
│   │   └── uploadMiddleware.ts         # File upload processing using Multer and Cloudinary
│   ├── routes/                         # API endpoint definitions
│   │   ├── adminRoutes.ts              # Admin-restricted endpoints for moderation
│   │   ├── authRoutes.ts               # Public and private authentication API endpoints
│   │   ├── chatRoutes.ts               # Real-time messaging and chat history endpoints
│   │   ├── matchRoutes.ts              # Matching system and queue management endpoints
│   │   ├── requestRoutes.ts            # Connection request and acceptance endpoints
│   │   └── userRoutes.ts               # Profile management and user metadata endpoints
│   ├── services/                       # Core business logic layer
│   │   ├── emailService.ts             # Email automation and transactional templates
│   │   ├── matchingService.ts          # Algorithmic logic for interest-based matching
│   │   ├── otpService.ts               # Secure OTP generation, storage, and verification
│   │   ├── queueService.ts             # Background task processing for matchmaking
│   │   └── tokenService.ts             # JWT generation and refresh token management
│   ├── socket/                         # Real-time communication
│   │   └── ...                         # Web Socket event handlers and real-time logic
│   ├── utils/                          # Shared helper functions
│   │   └── ...                         # Formatting, validation, and general utilities
│   └── server.ts                       # Main server entry point and Express app init
├── database_updates.sql                # SQL migration scripts and schema updates
├── nodemon.json                        # Development server auto-restart configuration
├── package.json                        # Backend dependencies, scripts, and metadata
├── tsconfig.json                       # TypeScript compiler settings for the server
└── ...                                 # Environment and local configuration files
```

DATABASE STRUCTURE :

Table Name	Description / Use Case
users	Core authentication table storing credentials, display names, and admin status.
verification_tokens	Manages temporary, unique security tokens for email verification and password resets.

sessions	Tracks active user logins, session IDs, and expiration timestamps for security.
user_blocks	Manages the blocklist to prevent contact between specific users securely.
user_profiles	Stores extended user data like avatars, interests, ratings, and ban status.
messages	Stores actual text payloads linked to senders and specific chat IDs.
reports	Logs user reports, including reasons and admin resolution status for moderation.
chats	Metadata container for persistent 1-on-1 direct messages between connected pairs.
connection_requests	Tracks friend requests sent between users and their current acceptance status.
matchmaking_queue	Active waiting room tracking users searching for a chat and their interests.
matchmaking_sessions	Records successful matched encounters, WebRTC room IDs, and session durations.

TECH STACK SUMMARY:

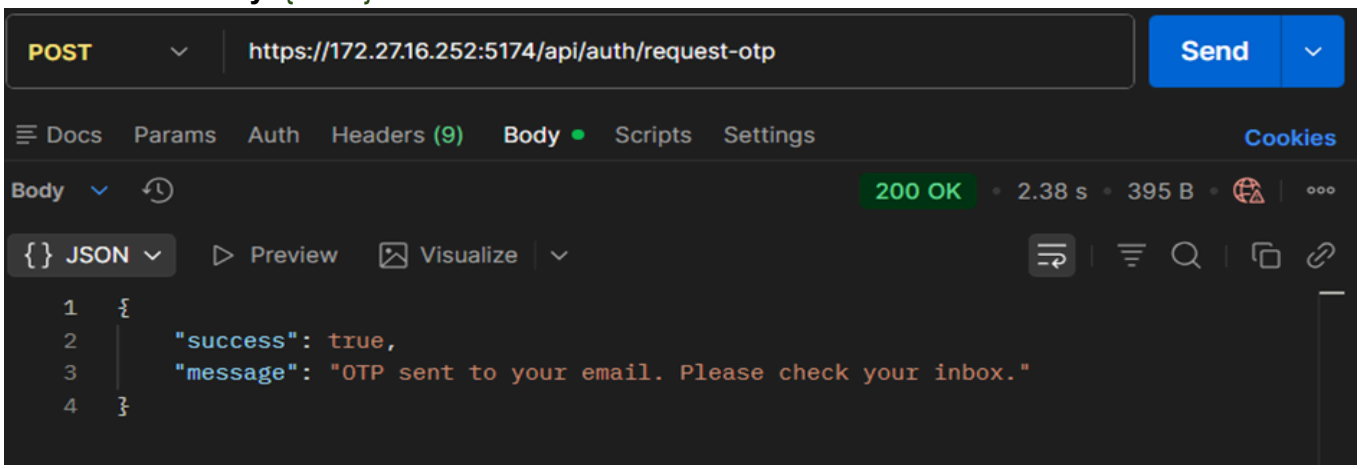
Layer	Technologies
Frontend	React 18, Vite, TypeScript, Tailwind CSS 4
Backend	Express.js, TypeScript, Node.js, Socket.IO
Database	PostgreSQL (pg), Redis (caching & queues)
Realtime	Socket.IO (WebSocket), WebRTC (video/audio)
Auth	JWT (access tokens), Bcrypt, sessions, OTP email verification
Storage	Cloudinary (avatar uploads via Multer)
Email	Nodemailer (SMTP: verification & password reset emails)
Styling	Tailwind CSS, CSS custom properties, Framer Motion animations

API Endpoints:

1. Authentication

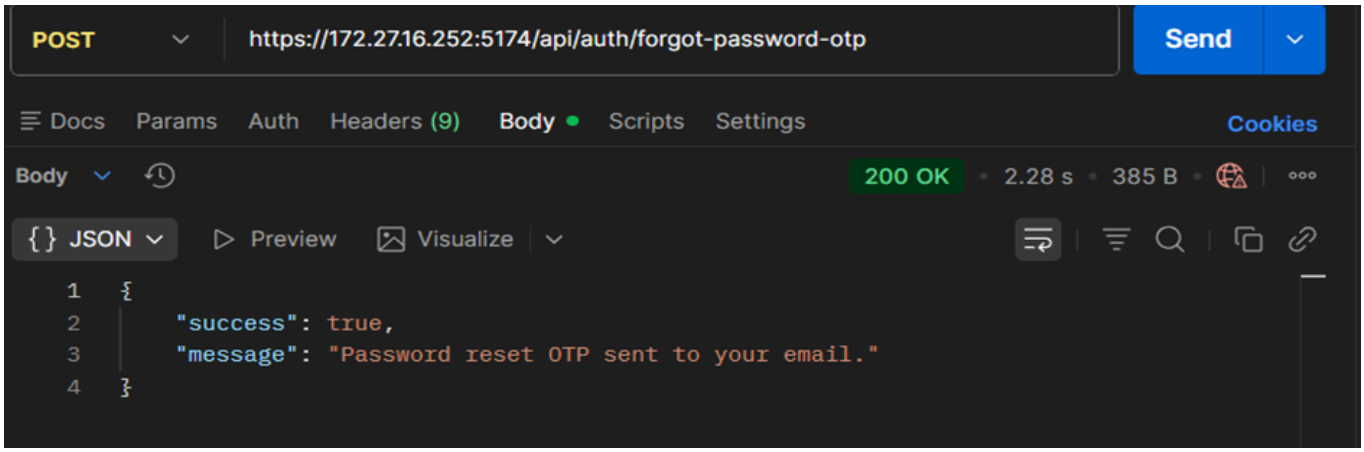
I. Registration

URL: auth/request-otp
Body: {email}



II. Forgot Password

URL: api/auth/forgot-password-otp
Method: POST
Body: {email}



III. Login

URL: api/auth/login

Method: POST

Body: {email, password}

POST https://172.27.16.252:5174/api/auth/login Send

Docs Params Auth Headers (9) Body Scripts Settings Cool

Body 200 OK 296 ms 902 B Save Response

{} JSON Preview Visualize


```
1  {
2      "success": true,
3      "message": "Login successful",
4      "data": {
5          "user": {
6              "id": 7,
7              "email": "krishvt23@iitk.ac.in",
8              "displayName": "KVT",
9              "verified": true
10         },
11         "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
               eyJ1c2VySWQiOjcsImVtYWlsIjoia3Jpc2h2dDIzQGlpdGsuYWMuaW4iLCJ2ZXJpZmllZCI6
               dHJ1ZSwiaWF0IjoxNzc0Nzg5Mzk5LCJleHAiOjE3NzUzOTQxOTl9.
               q1Eq9282TYIf5wGWXTOGrDb8-5ZOnabcapJp8_6e5Ow",
12         "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
               eyJ1c2VySWQiOjcsImVtYWlsIjoia3Jpc2h2dDIzQGlpdGsuYWMuaW4iLCJ2ZXJpZmllZCI6
               dHJ1ZSwiaWF0IjoxNzc0Nzg5Mzk5LCJleHAiOjE4MDYzMjUzOTl9.
               L2FNxHVFX-xqvIa90Ve4YdqwL3_mt4wnZVfniIvCIT0"
13     }
```

2. Match-Making Session

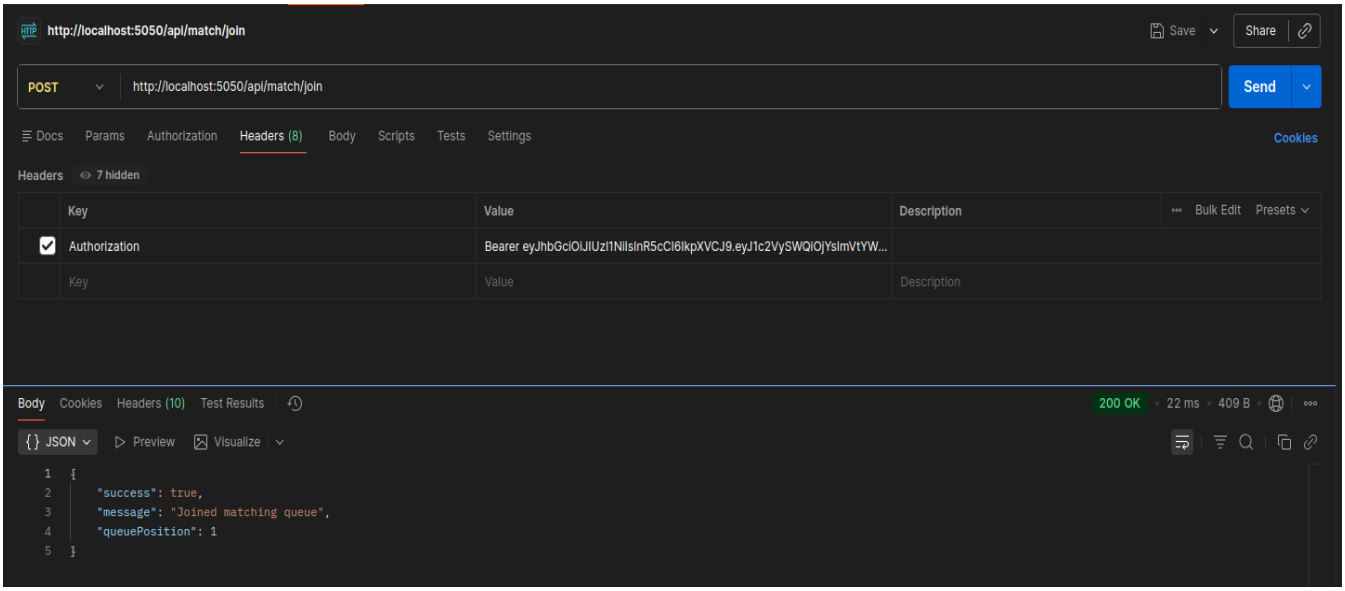
I. Joining the matchmaking queue:

URL: api/match/join

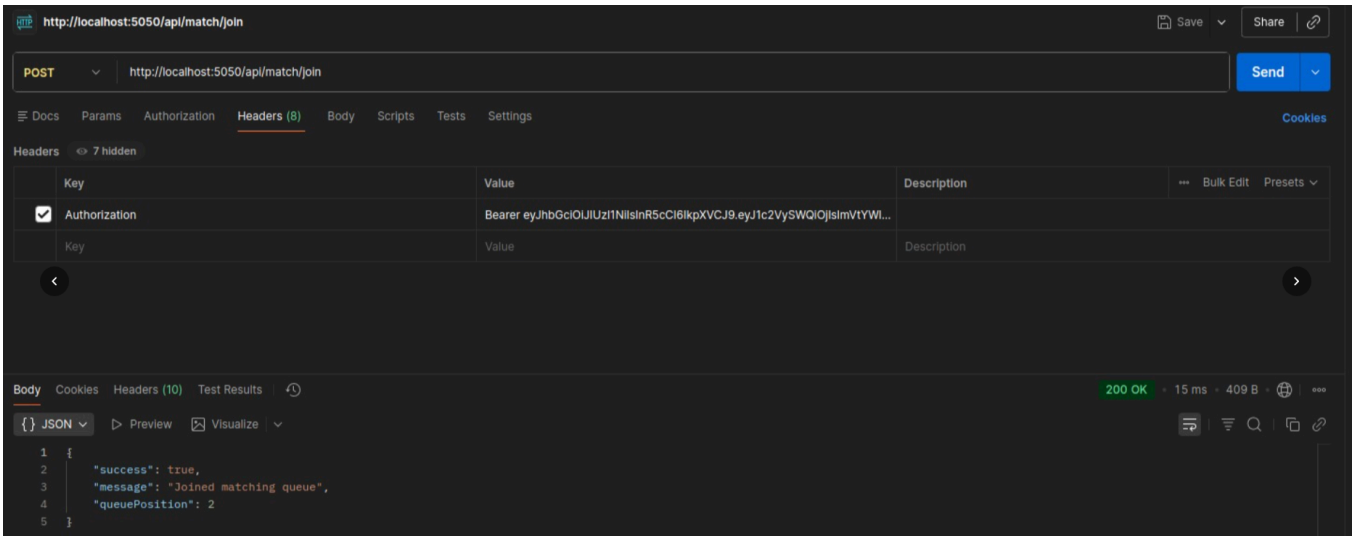
Method: POST

Header: { authtoken }

Status: For User 1



Status: For User 2



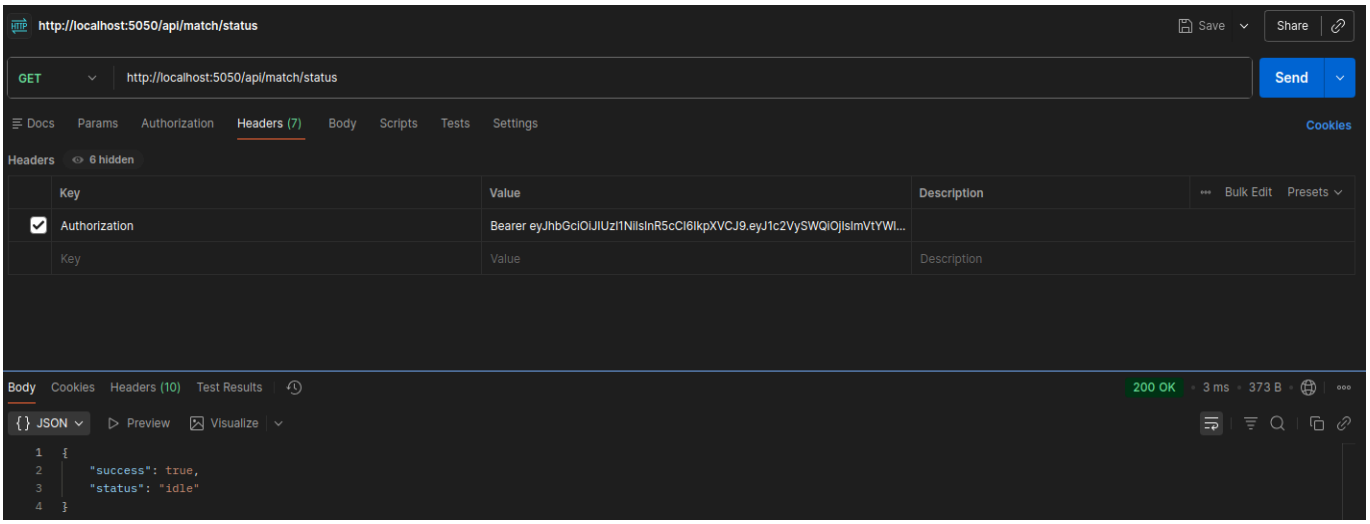
II. Status after joining the matchmaking queue:

URL: api/match/status

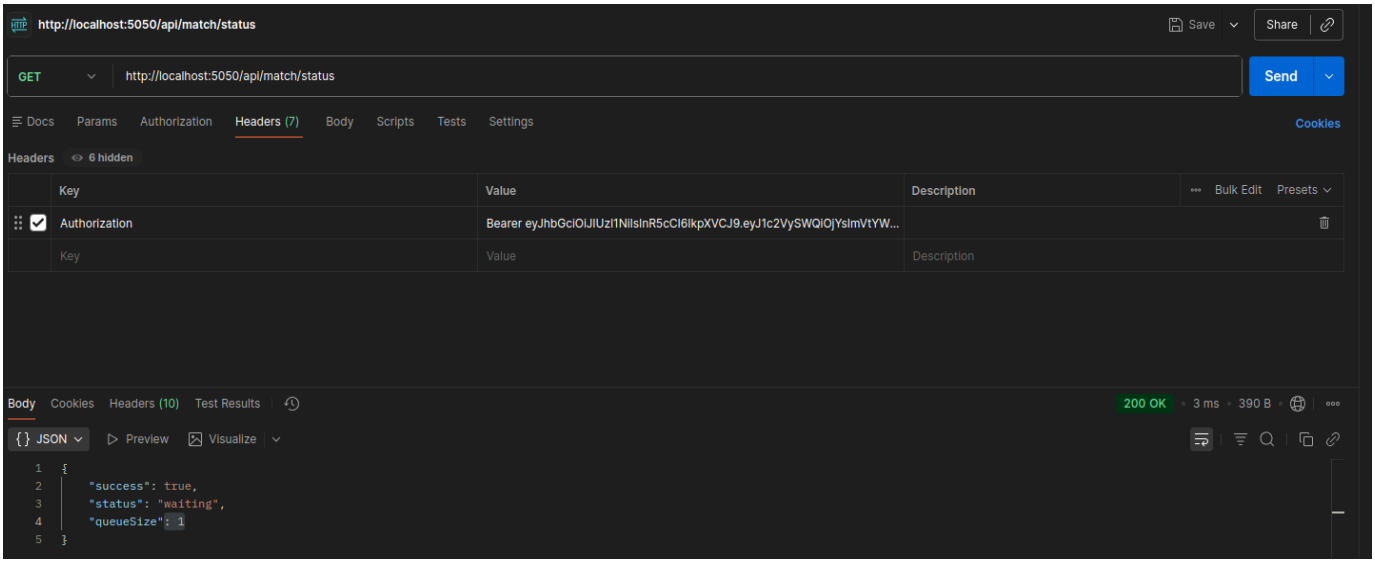
Method: GET

Header: { authtoken }

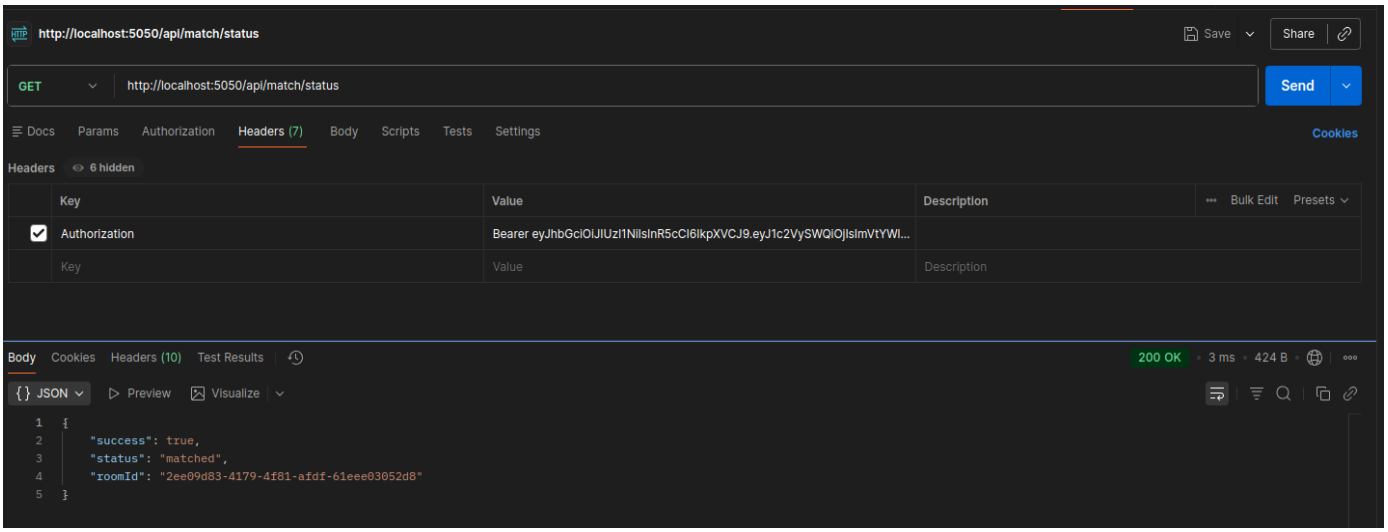
Status: For User 1: Before and After Joining the queue(state: Idle)



Status: For User 1: While waiting in the queue (state: WAITING)



For any invalid authtoken or incorrect UI, correct status “401” is returned by backend



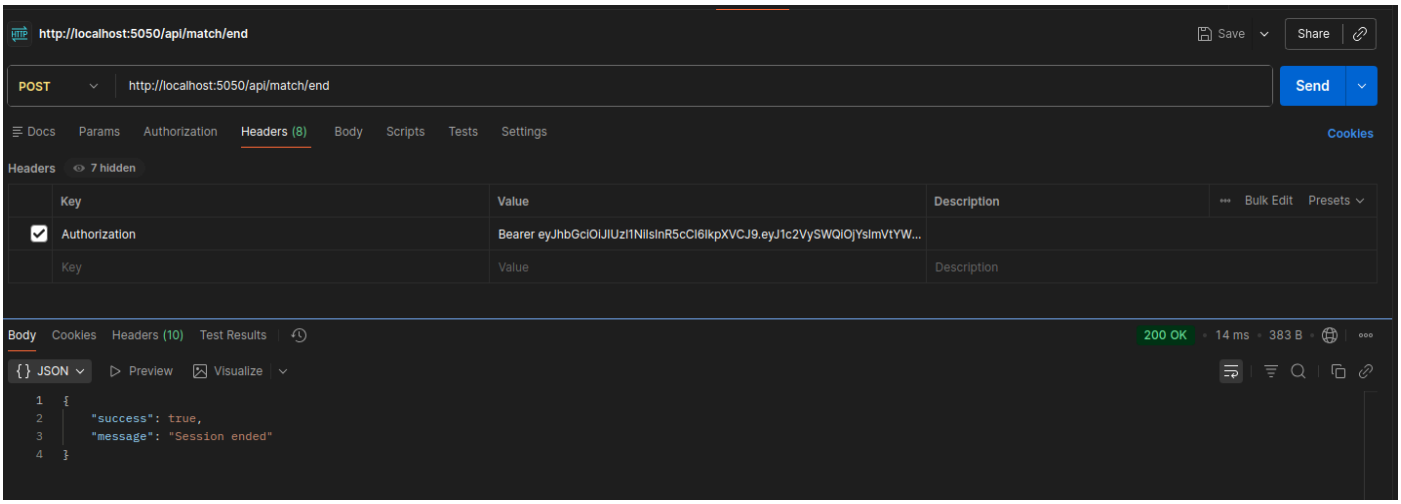
Status: For User 1: Matched with User 2 (state: MATCHED)

III. End the Video/Chat Session:

URL: api/match/end

Method: POST

Header: { authtoken }
Status: For User 1: Leaves the session



If the user is not in an active session, error code “400” is returned by the backend

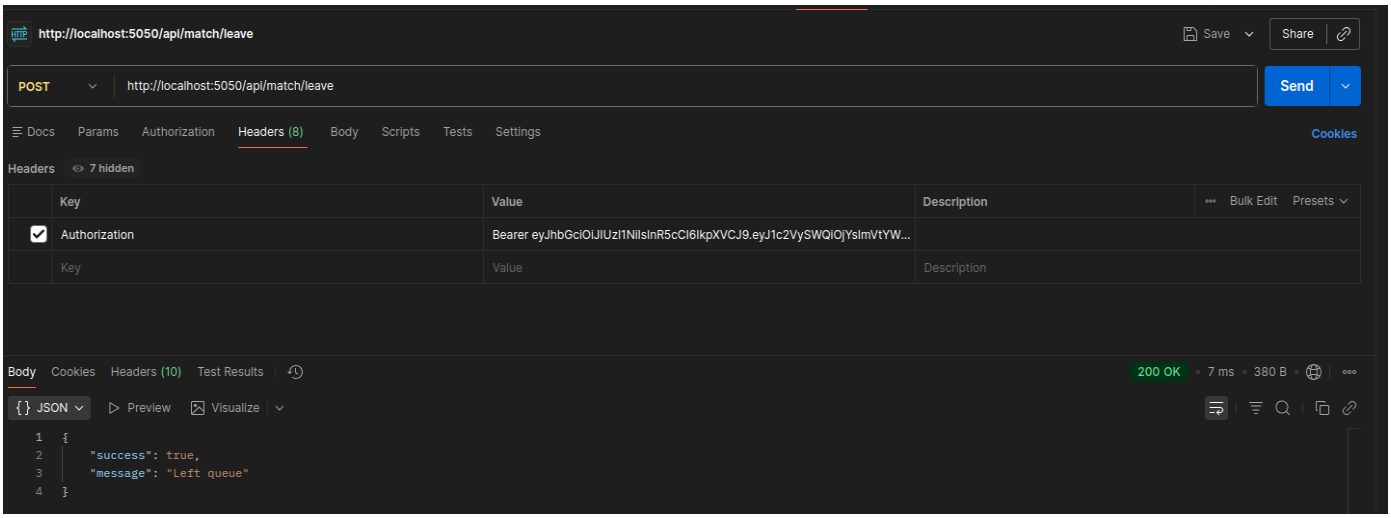
IV. Leaving the Queue:

URL: api/match/end

Method: POST

Header: { authtoken }

Status: For User 1: Leaves the matching making queue



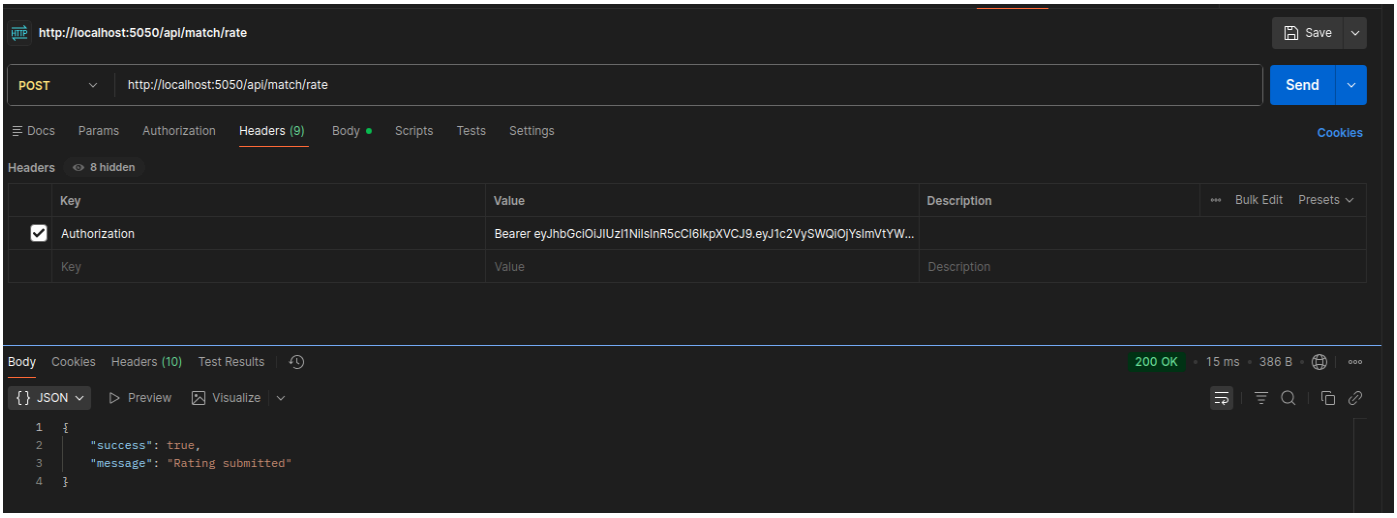
V. Rating Submission:

URL: api/match/end

Method: POST

Header: { authtoken }
Body: {{"roomId": "96a73849-ca55-40f7-8046-b29c03d5eabf", "rating": 5}

Status: For User 1: Leaves the matching making queue



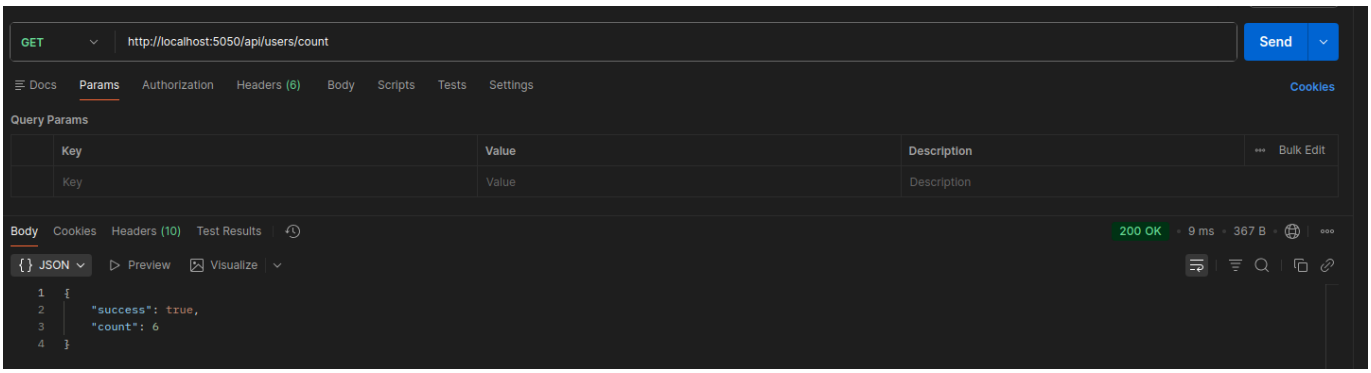
3. Users

I. Total Number of People in the queue:

URL: api/users/count

Method: GET

Status:

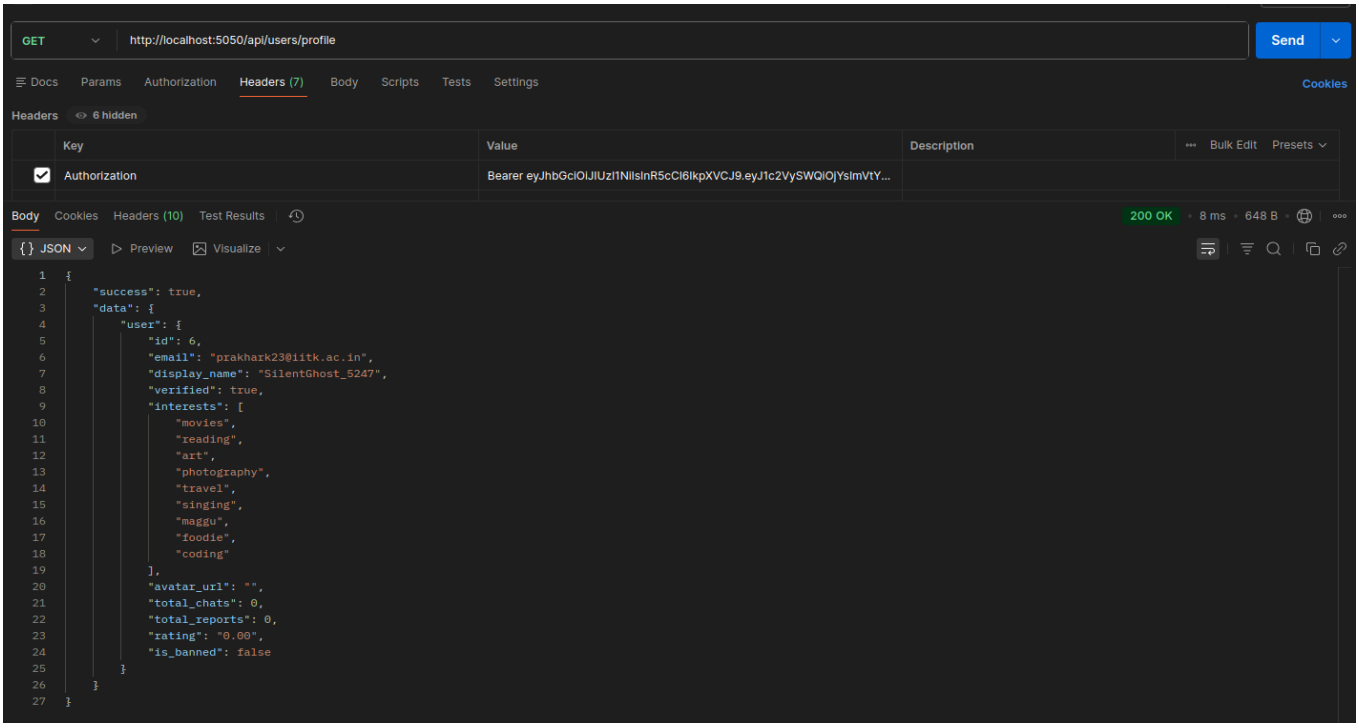


II. User's Profile:

URL: api/users/profile

Method: GET

Status: User 1 Data from his authorization token



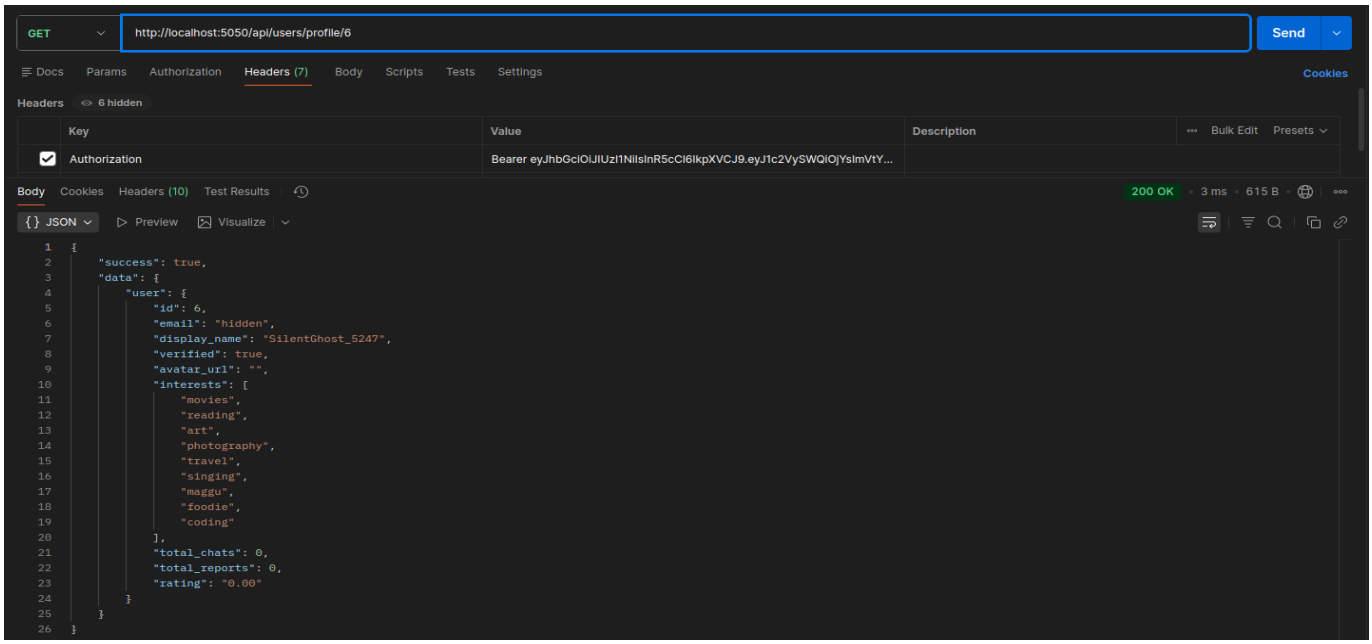
IV. See Others Anonymous Profile(Friends only):

URL: api/users/profile/1

Method: POST

Header: { authtoken }

Status: For User 1: See's anonymous details of friends



IV. Update Anonymous Profile:

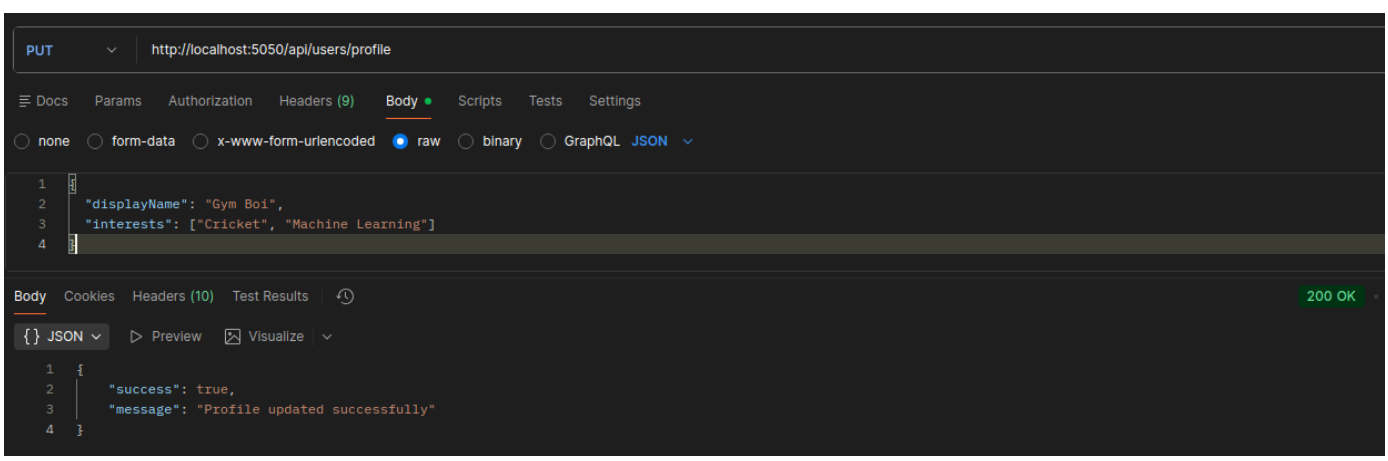
URL: api/users/profile

Method: PUT

Header: { authtoken }

Body : {{ "displayName": "Gym Boi", "interests": ["Cricket", "Machine Learning"] }

Status: For User 1: See's anonymous details of friends



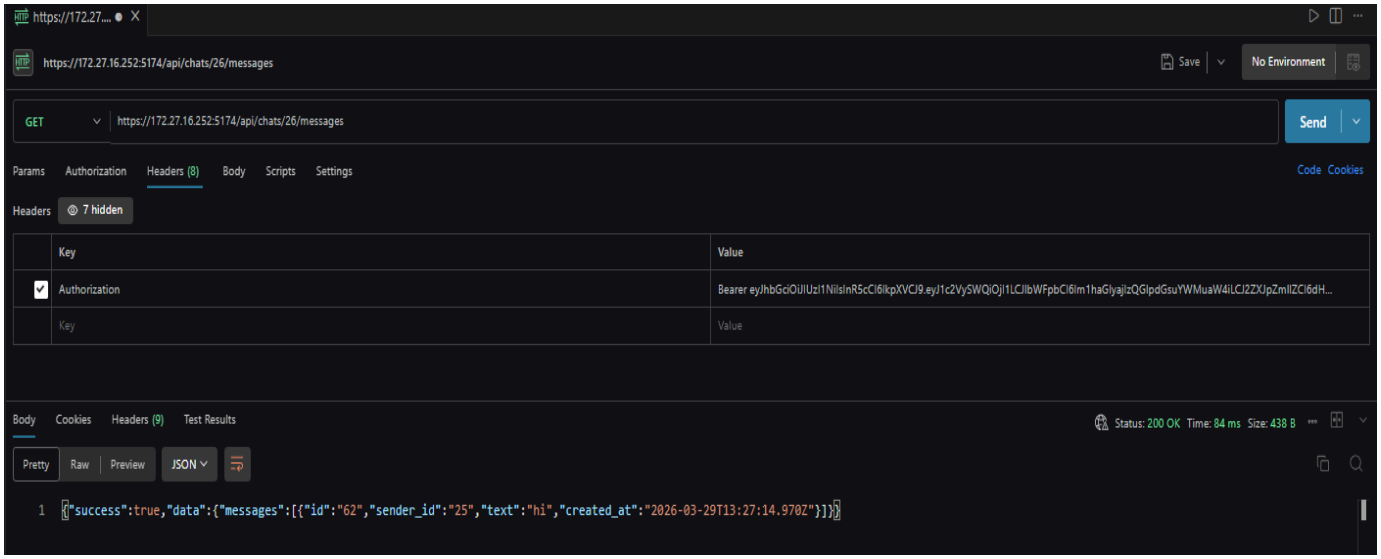
4. User Chats

I. Fetch Messages of a chat

URL: api/chats/:chat_id/messages

Method: GET

Header: { authtoken }

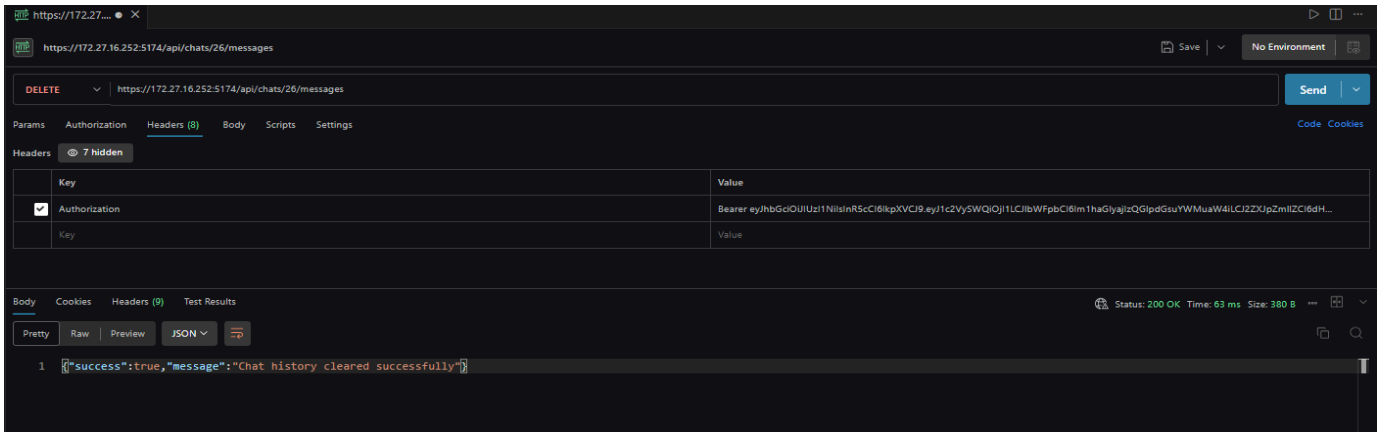


II. Delete messages in a chat

URL: api/chats/:chat_id/messages

Method: DELETE

Header: { authtoken }



III. Get All Chats

URL: api/chats/

Method: GET

Header: { authtoken }

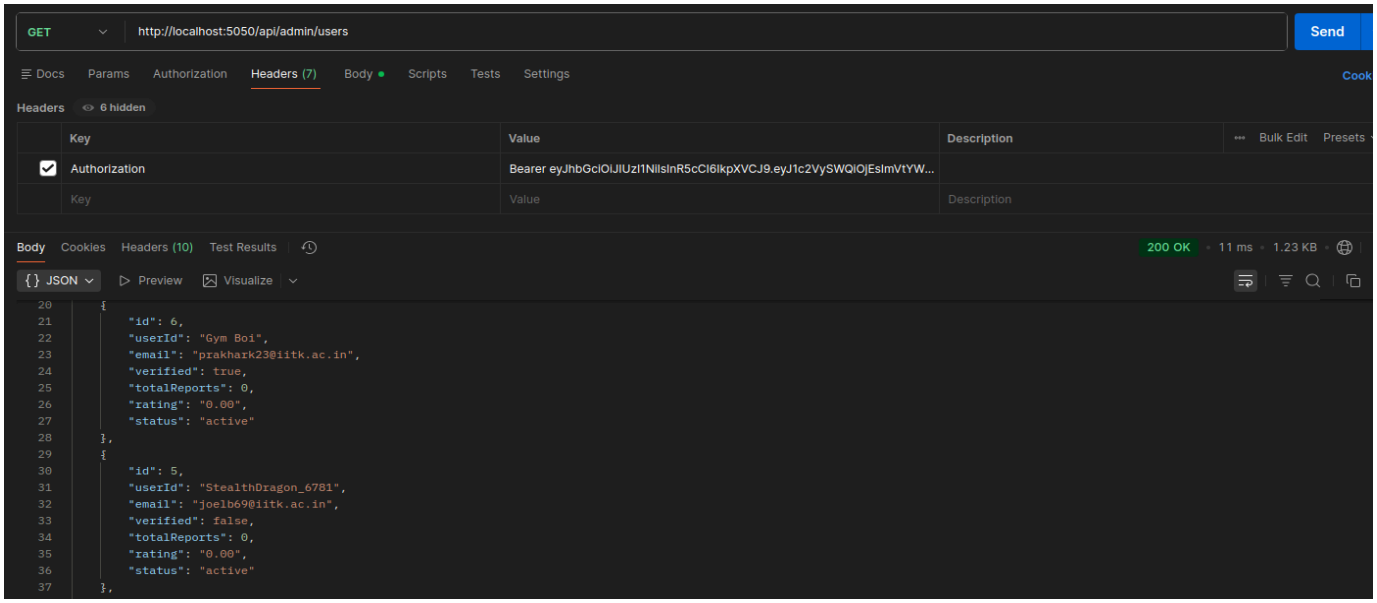
5. ADMIN ENDPOINTS

I. GET ALL USERS

URL: api/admin/users

Method: GET

Header: { authtoken }

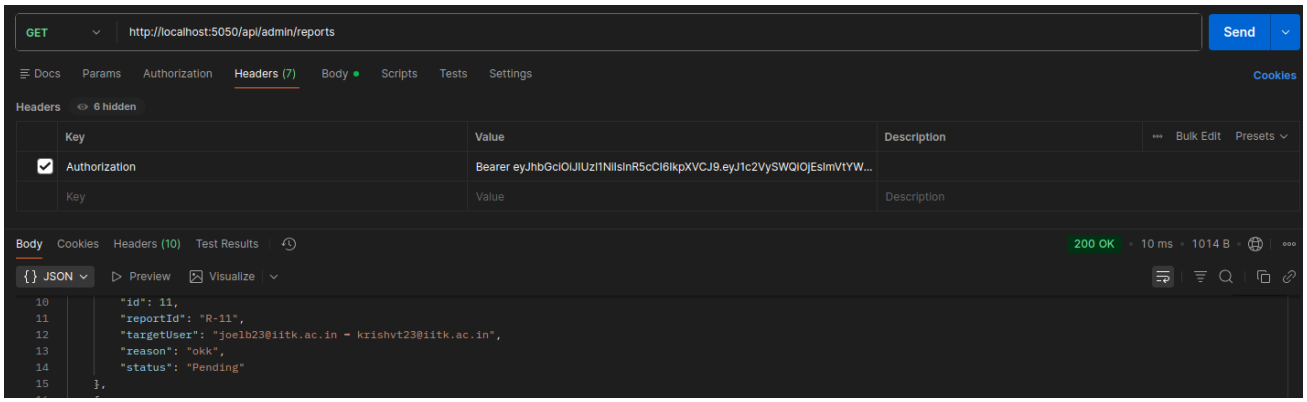


II. GET PENDING REPORTS

URL: api/admin/reports

Method: GET

Header: { authtoken }

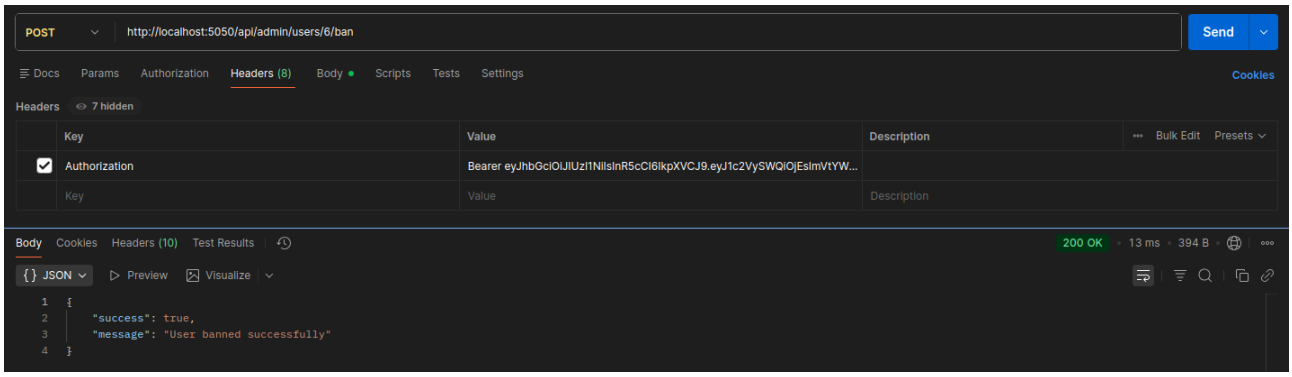


III . BAN A USER

URL: api/admin/users/:userId/ban

Method: POST

Header: { authtoken }



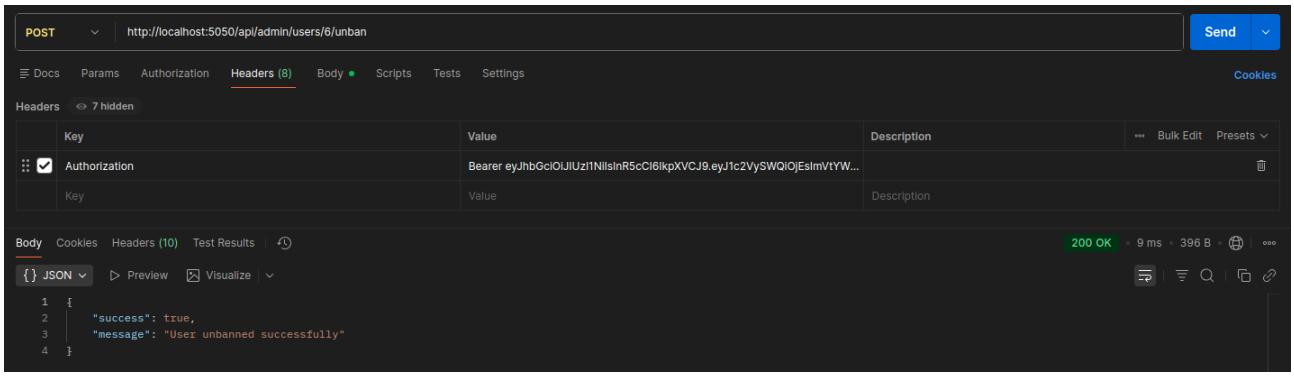
If user is not an admin then it gives “401” unauthorized access on accessing any of these endpoints, thus these are not visible to anyone other then the admin

IV . UNBAN A USER

URL: api/admin/users/:userId/unban

Method: POST

Header: { authtoken }



V . Update Report Status (Dismiss / Resolve)

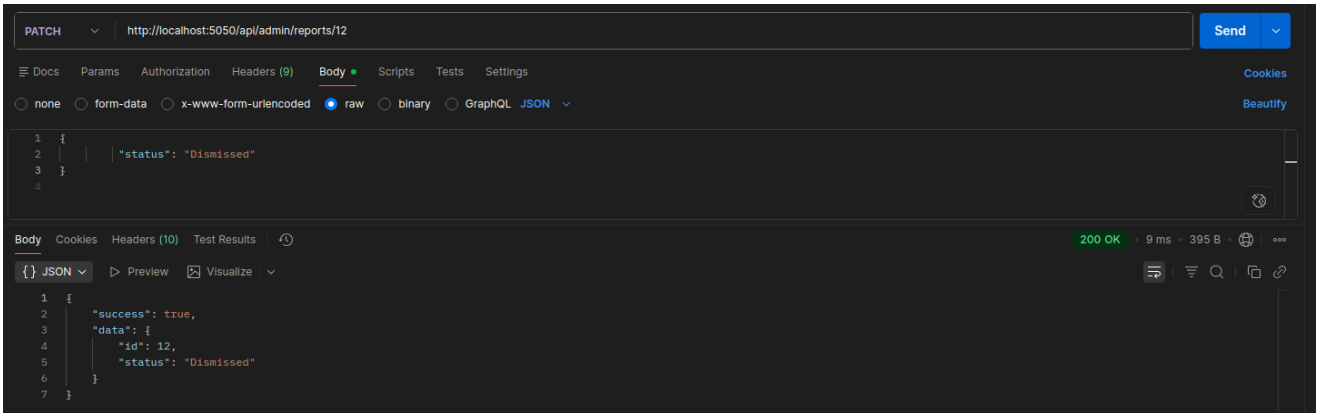
URL: api/admin/reports/:id

Method: PATCH

Header: { authtoken }

Body:

```
{
        "status": "Dismissed" // or "Resolved"
}
```



3 Completeness

IncognIITo is an anonymous, IITK-only video interaction platform designed to enable safe and meaningful one-to-one communication between users based on shared interests. The system integrates real-time matchmaking, session-based interaction, and moderation mechanisms to ensure both usability and safety while preserving user anonymity. Matchmaking uses a Jaccard similarity algorithm over user interest bitmaps, with a time-decay threshold to ensure users are matched within a bounded wait time.

User Features:

- IITK email-based registration with OTP verification ensures restricted access.
- Login, logout, and forgot password functionality with secure authentication.
- Anonymous profile setup using display name and profile picture.
- Selection and updating of interest tags for matchmaking.
- Entry into the matchmaking queue for pairing with compatible users.
- One-to-one session-based interaction between matched users.
- Real-time video communication using peer-to-peer connection.
- In-session real-time text chat functionality.
- Session termination using “Next” functionality.
- Post-session rating system (1–5 scale).
- Reporting mechanism for flagging inappropriate users.
- Sending, accepting, and declining Direct Message (DM) requests.
- Persistent direct messaging with accepted connections.

Admin Features:

- View all registered users.
- Ban users from accessing the platform.
- Review user reports and take appropriate action.
- Unban users and restore platform access.

Following Non Functional requirements have been achieved:

Safety Requirements:

• **Mandatory Authentication:** Access to the system requires users to register using their IITK email ID with OTP verification and password creation, ensuring only authorized users can enter the platform.

• **User Privacy:** User identity is protected by separating authentication data (email, credentials) from public profile information, ensuring that real identities are not exposed during interactions.

• **Controlled Interaction:** Users can only participate in sessions after successful authentication, preventing unauthorized access to live interactions.

• **Session Safety:** Active sessions are properly managed and can be terminated when required, ensuring that users are not left in unintended or insecure states.

Security Requirements:

• **Confidentiality:** All interactions between users are anonymized, and sensitive data like emails or credentials are never exposed through the application interface. This is enforced by using **JWTs** to identify users via unique, non-descriptive UIDs during **Socket.IO** sessions. Additionally, the backend controllers explicitly filter **PostgreSQL** query results to ensure only public metadata reaches the frontend.

• **Integrity:** Backend controls ensure that session states, user data, and administrative actions like banning or resolving reports are consistently maintained without corruption. This is achieved through **PostgreSQL transactions**, which enforce ACID properties and prevent partial updates during critical state changes. Additionally, **Redis** is utilized for real-time session tracking, ensuring that administrative status updates are instantly reflected across active **Socket.IO** connections.

• **Availability:** The system is designed to allow authenticated users to access matchmaking and session features reliably, ensuring uninterrupted interaction. To support this, **Redis** manages a high-availability matchmaking queue that persists even if individual backend workers restart. Furthermore, **Socket.IO** heartbeat mechanisms are used to automatically detect and recover dropped connections, maintaining a stable real-time environment for the **IIT Kanpur** community.

• **Access Control:** Role-based authorization ensures that administrative actions such as banning users, managing interest tags, and reviewing reports are restricted to authorized administrators only. This is implemented via **Express.js** middleware that validates **JWT roles** before granting access to protected API endpoints. By isolating administrative routes from the public API, we prevent unauthorized users from tampering with the platform's moderation state.

Software Quality Attributes:

• **Reliability:** The system ensures stable session handling and consistent backend operations during user interactions. This is achieved through **Socket.IO acknowledgment** patterns and robust **Node.js** error-handling middleware that prevents server crashes during malformed requests. Furthermore, **PostgreSQL** connection pooling ensures that the database remains responsive even during peak matchmaking periods.

• **Performance:** Efficient request handling and lightweight backend design enable smooth real-time matching and interaction. This is achieved by offloading the matchmaking queue to **Redis**, which allows for sub-millisecond lookups and atomic updates. Additionally, the **Node.js/Express** backend utilizes asynchronous I/O and optimized **PostgreSQL**indexing to minimize latency during concurrent user sessions.

•**Scalability:** The architecture supports handling multiple concurrent users and sessions effectively. This is achieved by using **Redis** for distributed state management, allowing the system to scale horizontally across multiple **Node.js** worker processes. Additionally, the **PostgreSQL** connection

pool is optimized to handle high-frequency queries, ensuring consistent performance as the **IITK community** grows.

• **Usability:** The interface is designed to provide intuitive navigation across login, matchmaking, interaction, and admin functionalities. This is achieved using **React** with **Tailwind CSS**, ensuring a responsive and consistent UI that adapts across mobile and desktop devices. Furthermore, **Vite's** fast HMR (Hot Module Replacement) and optimized build process ensure a low-latency, fluid experience during real-time sessions.

The Following Features are Planned for Future Updates:

• **Adaptive matchmaking with learning-based scoring:** Enhance the interest-based matching algorithm to incorporate session history, user feedback, and interaction patterns, moving beyond static interest similarity towards a dynamic scoring model that improves with usage.

• **Granular moderation controls:** Expand admin capabilities to support time-limited suspensions in addition to permanent bans, along with automated flagging of users who receive repeated reports, reducing the manual overhead of moderation at scale.

• **Scalable infrastructure for concurrent load:** Transition the matchmaking and session management services to a horizontally scalable architecture to support a significantly larger concurrent user base beyond the current IITK community, if the platform is extended in scope.

• **Session analytics and usage insights:** Introduce an analytics layer for administrators covering metrics such as average match quality, session duration, peak usage periods, and report resolution rates, enabling data-driven decisions about platform health.

Appendix A - Group Log

- We have a WhatsApp group for effective communication.
- We have created a private organization on GitHub so that we can work on the project with collaboration.
- We have made a comprehensive plan of which members of the team will handle different parts of the project like design, frontend, backend, and database etc.

Date/Time	Meet type	What was discussed in the meet?
01/03/26 17:30 - 22:00	Online Meet	Initial project planning and architecture discussion
05/03/26 20:00 - 22:00	Online Meet	Task distribution and module assignment
10/03/26 16:00 - 19:30	Offline Meet (Hall 2)	Backend and frontend integration discussion
15/03/26 21:00 - 01:00	Offline Meet(Hall 12)	Admin feature implementation and testing
18/03/26 21:00 - 01:00	Offline Meet(Hall 12)	Final document review and fixes